

EVIDENCE • CANVAS • PROSE

code

Single fenced code block as the slide's centerpiece.

One block, syntax lit, sized to be read from the back.

```
// The code slide holds ONE idea – a function, not a file.  
function fitsOnASlide(block) {  
  const lines = block.split('\n').length;  
  // Twelve lines reads from the back row; twenty is the wall.  
  return lines ≤ 12 ? 'readable' : lines ≤ 20 ? 'squinting' : 'split it';  
}
```

Twenty lines is the wall.

```
// A stress block at the ceiling: every line still earns its place.  
function stressTheFrame(lines) {  
  const budget = 20;           // the hard wall  
  const readable = 12;         // the comfort line  
  if (lines ≤ readable) return 'fine';  
  if (lines > budget) return 'split it'; // two slides beat one scroll  
  // Between comfort and the wall, trim ruthlessly:  
  // - drop imports and boilerplate  
  // - elide with ... what the point survives without  
  // - keep the line the talk is about  
  const keep = ['the signature', 'the branch', 'the return'];  
  return keep.join(' + ');  
}  
// The frame does not scroll. The audience does not squint.  
// What does not fit was never the point.
```

One block, syntax lit, sized to be read from the back.

```
// The code slide holds ONE idea – a function, not a file.  
function fitsOnASlide(block) {  
  const lines = block.split('\n').length;  
  // Twelve lines reads from the back row; twenty is the wall.  
  return lines ≤ 12 ? 'readable' : lines ≤ 20 ? 'squinting' : 'split it';  
}
```

One block, syntax lit, sized to be read from the back.

```
// The code slide holds ONE idea – a function, not a file.  
function fitsOnASlide(block) {  
  const lines = block.split('\n').length;  
  // Twelve lines reads from the back row; twenty is the wall.  
  return lines ≤ 12 ? 'readable' : lines ≤ 20 ? 'squinting' : 'split it';  
}
```

One block, syntax lit, sized to be read from the back.

```
// The code slide holds ONE idea – a function, not a file.  
function fitsOnASlide(block) {  
  const lines = block.split('\n').length;  
  // Twelve lines reads from the back row; twenty is the wall.  
  return lines ≤ 12 ? 'readable' : lines ≤ 20 ? 'squinting' : 'split it';  
}
```

When NOT to reach for code.

Comparing two versions

If you need before/after, use `compare-code` — it gives both snippets parallel framing. `code` is for a single snippet doing one job.

Code-as-decoration

A screenshot of an IDE or a snippet the audience cannot read defeats the layout. If the code is too long to legibly fit, the slide isn't a code slide — it's a content slide that talks about code.

No language hint

A bare fence renders as undifferentiated mono. Always tag the language so the highlighter and the reviewer both know what they are looking at.

RELATED COMPONENTS

See also.

`compare-code` — before/after snippet comparison

`diagram` — the architecture matters more than the code

`math` — the equation is the argument, not the
implementation

`content` — code is one piece of a longer prose explanation